Bolakaz Licensing

User Manual — Administrator & Developer Guide

Version 1.0 · July 2026

Table of Contents

1. System Overview
2. Architecture & URLs
3. Admin Panel (Filament)
    a. Logging In
    b. Managing Products
    c. Managing Price Tiers
    d. Generating Client Credentials
    e. Viewing License Codes
4. Purchase Flow (for End Users)
    a. The Purchase Link
    b. How a User Purchases a License
5. Redemption API (for Client Site Developers)
    a. How It Works
    b. API Request
    c. API Response
    d. Example: PHP / Laravel
    e. Example: Node.js / JavaScript
    f. Example: Python
6. Adding a New Product (Zero Code Changes)
7. Production Deployment
8. Troubleshooting & FAQ

1 — System Overview

The **Bolakaz Licensing** system is a central platform that handles software license sales and validation across multiple products. Instead of every client site holding its own Paystack keys and managing its own code generation, they all use this single service.

What It Does

- **Sells license codes** for any number of products (Stock-Taking, CRM, POS, etc.)
- **Validates codes** when client sites redeem them
- **Prevents fraud** — server-computed pricing, amount cross-checking, race-condition protection
- **Requires zero code changes** to add a new product — just add it in the admin panel

The Three Roles

Role	Who	What They Do
Admin	You (Bolakaz owner)	Create products, set pricing, generate client credentials, view license codes
Client Site	Shop owners / site admins	Put the purchase link on their site, call the redeem API when users enter codes
End User	Customers of client sites	Visit the purchase link, pay via Paystack, receive a license code, enter it on the client site

2 — Architecture & URLs

Production URLs

What	URL	Hosted On
Purchase pages (user-facing)	`https://teslim.unibooks.com.ng/purchase/{slug}`	Vercel
API server (backend)	`https://api.teslim.unibooks.com.ng`	Your VPS
Admin panel	`https://api.teslim.unibooks.com.ng/admin`	Your VPS



```
|  ├── POST /api/webhook/paystack    Paystack call |
|  └── /admin                        Filament      |
└──────────────────────────────────────────────────┘
```

3 — Admin Panel (Filament)

The admin panel is where you manage everything. It lives at:

```
https://api.teslim.unibooks.com.ng/admin
```

3.1 — Logging In

Use the credentials created during deployment:

Field	Value
Email	`admin@bolakaz.ng`
Password	(set during deployment)

You will see the dashboard with stats: active products, total codes issued, lifetime revenue.

3.2 — Managing Products

Navigate to **Products** in the sidebar. Each row is a licensable product.

Creating a Product

1. **Click "New Product"**
   Fill in the form:

Field	Example	Notes
Name	Bolakaz Stock-Taking	Display name shown on purchase page
Slug	bolakaz-stocktaking	URL-safe ID; becomes `/purchase/bolakaz-stocktaking`
Code Prefix	BLKZ	4 letters; all license codes for this product start with this

Description	Stock-taking & inventory management...	Optional, shown on purchase page
Daily Rate	2000.00	Base price in Naira per day
Is Active	ON	Inactive products are hidden from the purchase page

2 **Click "Create"**
The product is now live at `https://teslim.unibooks.com.ng/purchase/bolakaz-stocktaking`.

3.3 — Managing Price Tiers

Navigate to **Price Tiers**. Each tier belongs to one product.

Creating Tiers

Tier Key	Duration	Discount	Use Case
`fixed_3day`	3 days	5%	Short trial
`fixed_7day`	7 days	10%	Weekly pass
`fixed_30day`	30 days	20%	Monthly pass
`custom`	User picks	0%	Flexible duration at base rate

Pricing formula: `daily_rate × duration_days × (1 − discount_pct / 100)`

For the "custom" tier, set **Duration Days** to empty and **Discount %** to 0. The user will enter how many days they want on the purchase page, and the price will be `daily_rate × custom_days` with no discount.

3.4 — Generating Client Credentials

Navigate to **Client Credentials**. Each credential grants one client site the ability to redeem codes for one specific product.

> ⚠️ **Critical: The API secret is shown ONLY ONCE.**
> After creating a credential, Filament will display the plaintext secret in a notification. Copy it immediately. It is stored as a bcrypt hash in the database and cannot be recovered later. If lost, you must create a new

credential.

Creating a Credential

1. **Click "New Client Credential"**

Field	Example
Product	Bolakaz Stock-Taking
Client Name	ShopName - shopname.ng
Domain	shopname.ng
Is Active	ON

2. **Click "Create"**
   Filament auto-generates the `client_key` and `api_secret`. Two notifications appear:
   - **Success notification:** shows the `client_key`
   - **Warning notification:** shows the `api_secret` in plaintext — copy it NOW

3. **Give these two values to the client site owner:**

```
X-Client-Key:    SHOPNAME_aB3xYz ... 9fG2
X-Client-Secret:  f8a2c1e9d7b6 ... 3f5e1a (40 characters)
```

   They will use these in their server-side code to call the redemption API. These values go in their `.env` file — never in frontend code.

> 💡 **One credential = one product + one client site.**
> If a site uses two products (e.g., Stock-Taking AND CRM), create two separate credentials — one for each product. This way, if a credential is compromised, only one product on one site is affected.

3.5 — Viewing License Codes

Navigate to **License Codes**. This is a read-only view showing every code ever generated, filterable by product and status (unused / redeemed). You can see:

- Which code was redeemed by which client site

- When it was redeemed
- The payment reference and amount paid

4 — Purchase Flow (for End Users)

4.1 — The Purchase Link

Each product has a single, permanent purchase URL:

```
https://teslim.unibooks.com.ng/purchase/{product-slug}
```

Examples:

Product	Purchase URL
Bolakaz Stock-Taking	`https://teslim.unibooks.com.ng/purchase/bolakaz-stocktaking`
Bolakaz CRM (future)	`https://teslim.unibooks.com.ng/purchase/bolakaz-crm`

Client sites put this link wherever they want users to buy a license — a "Buy License" button, a paywall page, a pricing card, an email, anywhere.

4.2 — How a User Purchases a License

1. **User visits the purchase link**
   They see the product name, description, daily rate, and tier options (e.g., 3 Days / 7 Days / 30 Days / Custom). Prices are computed server-side from the tiers you configured in Filament.

2. **User selects a plan and enters their email**
   For fixed tiers: clicks the tier card. For custom: enters number of days. The total price updates live.

3. **User clicks "Pay with Paystack"**
   A Paystack popup opens. The user completes payment with card, bank transfer, or USSD. They never leave the page.

4. **Payment confirmed — code appears**
   The user sees a success page with their license code displayed prominently and a "Copy" button. The code is also emailed to them.

5. **User enters the code on the client site**
   The client site's backend calls the redemption API to validate the code. If valid, the license is activated for the purchased duration.

> 🔒 **Security note:**
>
> The price is always computed on the server. The client never sends an amount — the Laravel API calculates it using the product's daily rate and tier configuration. On verification, the amount Paystack confirms is cross-checked against the server-computed amount. If they don't match, no code is issued.

5 — Redemption API (for Client Site Developers)

This section is for the developer who maintains the client site. Give them this section of the manual along with their credentials.

5.1 — How It Works

1. User enters a license code on the client site (a form or input field)
2. The client site's **backend** sends the code to the redemption API
3. The API responds with `valid: true` or `valid: false`
4. If valid, the client site activates the license for the returned duration

> ⚠️ **Never call this API from frontend JavaScript.**
>
> Your `api_secret` would be exposed to anyone who opens DevTools. Always call the redemption API from your server-side code (PHP, Node, Python, etc.).

5.2 — API Request

Item	Value
Method	`POST`
URL	`https://api.teslim.unibooks.com.ng/api/redeem-code`

Headers	`X-Client-Key: {your_client_key}`
	`X-Client-Secret: {your_api_secret}`
	`Content-Type: application/json`

Body	JSON: `{ "code": "...", "domain": "..." }`

Request Body Fields

Field	Type	Required	Description
`code`	string	Yes	The 19-character license code (e.g., `BLKZ-K3M9-XQ2P-4RTN`)

`domain`	string	Yes	The domain of the site redeeming the code (for your records)

5.3 — API Response

Success (200 OK)

```
{
  "valid": true,
  "duration_days": 7
}
```

Invalid Credentials (401 Unauthorized)

```
{
  "valid": false,
  "reason": "invalid credentials"
}
```

Invalid or Already Redeemed Code (422 Unprocessable)

```
{
  "valid": false,
  "reason": "License code not found, already redeemed, or not valid for this product."
}
```

Rate Limited (429 Too Many Requests)

More than 5 attempts per minute from the same IP. Wait and retry.

5.4 — Example: PHP / Laravel

```
<?php

use Illuminate\Support\Facades\Http;

function redeemLicenseCode(string $code, string $domain): array
{
    $response = Http::withHeaders([
        'X-Client-Key'    => env('BOLAKAZ_CLIENT_KEY'),
        'X-Client-Secret' => env('BOLAKAZ_CLIENT_SECRET'),
        'Content-Type'    => 'application/json',
```

```
    ])->post('https://api.teslim.unibooks.com.ng/api/redeem-code', [
        'code'   => strtoupper(trim($code)),
        'domain' => $domain,
    ]);

    return $response->json();
}

// ── Usage in a controller ──

public function activateLicense(Request $request)
{
    $request->validate([
        'license_code' => 'required|string|size:19',
    ]);

    $result = redeemLicenseCode(
        $request->input('license_code'),
        $request->getHost(),
    );

    if ($result['valid'] ?? false) {
        $days = $result['duration_days'];
        $expiresAt = now()->addDays($days);

        // Store the license activation in your database
        // License::create([ ... ]);

        return back()->with('success', "License activated for {$days} days!");
    }

    return back()->withErrors([
        'license_code' => $result['reason'] ?? 'Invalid license code.',
    ]);
}
```

Your .env file

```
BOLAKAZ_CLIENT_KEY=SHOPNAME_aB3xYz ... 9fG2
BOLAKAZ_CLIENT_SECRET=f8a2c1e9d7b6 ... 3f5e1a
```

5.5 — Example: Node.js / JavaScript

```
// Server-side only — never expose BOLAKAZ_CLIENT_SECRET to the browser

const BOLAKAZ_CLIENT_KEY = process.env.BOLAKAZ_CLIENT_KEY;
const BOLAKAZ_CLIENT_SECRET = process.env.BOLAKAZ_CLIENT_SECRET;

async function redeemLicenseCode(code, domain) {
  const response = await fetch(
```

```
    'https://api.teslim.unibooks.com.ng/api/redeem-code',
    {
      method: 'POST',
      headers: {
        'X-Client-Key': BOLAKAZ_CLIENT_KEY,
        'X-Client-Secret': BOLAKAZ_CLIENT_SECRET,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        code: code.trim().toUpperCase(),
        domain: domain,
      }),
    },
  );

  return response.json();
}

// ── Usage in an Express route ──

app.post('/activate-license', async (req, res) => {
  const { licenseCode } = req.body;
  const result = await redeemLicenseCode(licenseCode, req.hostname);

  if (result.valid) {
    // Activate the license for result.duration_days
    res.json({ success: true, days: result.duration_days });
  } else {
    res.status(422).json({ success: false, message: result.reason });
  }
});
```

5.6 — Example: Python

```
import os
import requests

BOLAKAZ_CLIENT_KEY = os.environ["BOLAKAZ_CLIENT_KEY"]
BOLAKAZ_CLIENT_SECRET = os.environ["BOLAKAZ_CLIENT_SECRET"]

def redeem_license_code(code: str, domain: str) -> dict:
    response = requests.post(
        "https://api.teslim.unibooks.com.ng/api/redeem-code",
        headers={
            "X-Client-Key": BOLAKAZ_CLIENT_KEY,
            "X-Client-Secret": BOLAKAZ_CLIENT_SECRET,
            "Content-Type": "application/json",
        },
        json={
            "code": code.strip().upper(),
            "domain": domain,
        },
```

```
    )
    return response.json()


# ── Usage in a Flask/Django view ──

def activate_license(request):
    code = request.POST.get("license_code")
    result = redeem_license_code(code, request.get_host())

    if result.get("valid"):
        days = result["duration_days"]
        # Activate the license for `days` days
        return {"success": True, "days": days}
    else:
        return {"success": False, "message": result.get("reason", "Invalid code")}
```

6 — Adding a New Product (Zero Code Changes)

This is the core design principle: onboarding a brand new product requires **no code changes and no deployment**. Everything is data-driven.

1. **Create the product in Filament**
   Admin → Products → New Product. Fill in name, slug, code prefix, daily rate.

2. **Add its price tiers**
   Admin → Price Tiers → New. For each tier, select the product and set the tier key, duration, and discount.

3. **Generate client credentials for each site**
   Admin → Client Credentials → New. One credential per (product + client site). Give the `client_key` and `api_secret` to the site owner.

4. **Give the site owner the purchase link**

   ```
   https://teslim.unibooks.com.ng/purchase/{product-slug}
   ```

   The React app automatically loads the product and its tiers from the API. The purchase page works instantly — no frontend changes needed.

That's it. Four steps. No code. No deploy. The purchase page, tier display, price calculation, Paystack integration, code generation, redemption API, and Filament admin all work for any product you add.

> 💡 **Pro tip:**
> Each product can have a completely different pricing structure. One product might offer 3/7/30-day tiers at ₦2,000/day with discounts, while another offers 14/30/90-day tiers at ₦5,000/day. Configure it in Filament and it just works.

7 — Production Deployment

DNS Setup

Record	Type	Value
`teslim.unibooks.com.ng`	A	Vercel IPs (portfolio)
`api.teslim.unibooks.com.ng`	A	Your VPS IP

VPS Setup (aaPanel)

1. Create a new site in aaPanel for `api.teslim.unibooks.com.ng`
2. Point the document root to `/www/wwwroot/licensing-app/public`
3. Issue a free Let's Encrypt SSL certificate
4. Set the production `.env` values (see below)
5. Run `php artisan migrate --force`
6. Create the admin user: `php artisan make:filament-user`

Production .env (on VPS)

```
APP_NAME="Bolakaz Licensing"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://api.teslim.unibooks.com.ng

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=licensing
DB_USERNAME=root
DB_PASSWORD=(your-db-password)

CORS_ALLOWED_ORIGIN=https://teslim.unibooks.com.ng

PAYSTACK_SECRET_KEY=sk_live_xxxxxxxx
PAYSTACK_PUBLIC_KEY=pk_live_xxxxxxxx
```

```
MAIL_MAILER=smtp
MAIL_HOST=(your-smtp-host)
MAIL_PORT=587
MAIL_USERNAME=(your-smtp-username)
MAIL_PASSWORD=(your-smtp-password)
MAIL_FROM_ADDRESS=noreply@bolakaz.ng
MAIL_FROM_NAME="Bolakaz Licensing"
```

Vercel Setup

In the Vercel dashboard for your portfolio project, add an environment variable:

Key	Value
`VITE_API_BASE`	`https://api.teslim.unibooks.com.ng`

Redeploy the portfolio after setting this variable.

Paystack Webhook

In your Paystack dashboard, set the webhook URL to:

```
https://api.teslim.unibooks.com.ng/api/webhook/paystack
```

This ensures license codes are generated even if the user closes their browser after payment (the webhook is the fallback).

8 — Troubleshooting & FAQ

Common Issues

"The purchase page shows 'Product not found'"

- Check that the product `is_active` is ON in Filament
- Verify the slug in the URL matches exactly (case-sensitive)
- Check that `VITE_API_BASE` is set correctly in Vercel and the portfolio was redeployed

"Payment goes through but no code appears"

- Check the Laravel logs on the VPS: `storage/logs/laravel.log`
- Verify the Paystack webhook URL is set correctly in the Paystack dashboard
- The webhook is the fallback — if the browser callback fails, the webhook should still generate the code

"Redeem API returns 'invalid credentials'"

- Verify the `client_key` and `api_secret` match what was generated
- Check that the credential's `is_active` is ON in Filament
- Remember: the `api_secret` is shown only once. If lost, create a new credential

"Code redeems successfully but unlocks the wrong product"

This can't happen. The redemption API checks that the code's `licensable_product_id` matches the credential's `licensable_product_id`. A Stock-Taking credential can never redeem a CRM code.

FAQ

Can a single client site use multiple products?

Yes. Create one credential per product for that site. They'll have multiple sets of `client_key` + `api_secret` — one per product. When redeeming a code, they use the credential that matches the product the code belongs to.

What happens if someone tries to use the same code twice?

The database transaction with `lockForUpdate()` prevents this. Even if two requests arrive simultaneously for the same code, only one succeeds. The second gets `{ "valid": false }`.

Can I change pricing after codes have been sold?

Yes. Changing a product's `daily_rate` or a tier's `discount_pct` only affects **future** purchases. Existing codes are unaffected. To remove a tier without deleting it, toggle `is_active` to OFF.

What if Paystack is down?

Users will see a "Payment initialization failed" error. The purchase flow won't proceed. Once Paystack recovers, everything works again. No codes can be lost because codes are only generated after confirmed payment.

Is my api_secret safe?

Yes. It's stored as a bcrypt hash in the database (not plaintext). The middleware uses `Hash::check()` to verify it — the same secure approach Laravel uses for user passwords. The `client_key` is a public identifier (like a username) and is stored in plaintext for fast indexed lookup.
